KMedoids Clustering

Another variant that is robust to the outliers is *K*-Medoids. Let's implement *K*-Medoids from scratch as well to understand it's fundamentals.

In [ ]:

```
import matplotlib
import matplotlib.pyplot as plt
import numpy as np
```

In [ ]:

```
np.random.seed(100) # change seed value and see for different random init

plt.rcParams['figure.figsize'] = (10, 5)
```

In [ ]:

```
from sklearn.datasets import make_blobs

n_samples = 100  # number of samples to generate
random_state = 100  # seed value

## Generate noisy data points
X_noisy, _ = make_blobs(
    n_samples=n_samples,
    random_state=random_state,
    cluster_std=[1.5, 1, 1, 10],
    centers=4,
)

## Generate clean data points
X_clean, _ = make_blobs(
    n_samples=n_samples, random_state=random_state, cluster_std=[1.5, 1, 1], centers=3
)

print(X_clean.shape, X_noisy.shape)
```

```
(100, 2) (100, 2)
```

K-Medoids from Scratch

In [ ]:

```
## Custom KMedoids implementation from scratch


class KMedoids:
    def __init__(self, n_centers, threshold=1e-4):
        self.n_centers = n_centers
        self.threshold = threshold
        self.centers = None
        self.max_iter = 10

    def __distance(self, data1, data2):
        """ """

        return np.sqrt(np.sum((data1 - data2) ** 2))

    def __medoids_cost(self, centers):
        """ """

        X = self.data
        dist_mat = np.zeros((len(X), len(centers)))

        for j in range(len(centers)):
            center = X[centers[j], :]
            for i in range(len(X)):
                if i == centers[j]:
                    dist_mat[i, j] = 0.0
                else:
                    dist_mat[i, j] = self.__distance(X[i, :], center)

        mask = np.argmin(dist_mat, axis=1)
        members = np.zeros(len(X))
        costs = np.zeros(len(centers))
        for i in range(len(centers)):
            mem_id = np.where(mask == i)
            members[mem_id] = i
            costs[i] = np.sum(dist_mat[mem_id, i])
        return members, np.sum(costs)

    def __get_init_centers(self):
        """ """

        init_ids = []
        while len(init_ids) < self.n_centers:
            _ = np.random.randint(0, self.data.shape[0])
            if not _ in init_ids:
                init_ids.append(_)
        return init_ids

    def fit(self, X):
        """

        """
        self.data = X
        self.centers = self.__get_init_centers()
        centers = self.centers
        members, tot_cost = self.__medoids_cost(centers)
        cc, SWAPED = 0, True
        while True:
            SWAPED = False
```

```
            for i in range(self.data.shape[0]):
                if not i in centers:
                    for j in range(len(centers)):
                        centers_ = np.copy(centers)
                        centers_[j] = i
                        members_, tot_cost_ = self.__medoids_cost(centers_)

                        if tot_cost_ - tot_cost < self.threshold:
                            members, tot_cost = members_, tot_cost_
                            centers = centers_
                            SWAPED = True
            if cc > self.max_iter:
                break
            cc += 1
        return centers, tot_cost, members
```

Like our custom `KMeans`, we have built a class for `KMedoids` as well. We have a public `fit` method to train our model. Let's fit our noisy data and see how this variant performs.

In [ ]:

```
kms = KMedoids(3)
centers, cost, members = kms.fit(X_noisy)
```

Here also we have initialized 3 cluster centroids and trained our noisy data. Let's visualize the output of `KMedoids` below:

In [ ]:

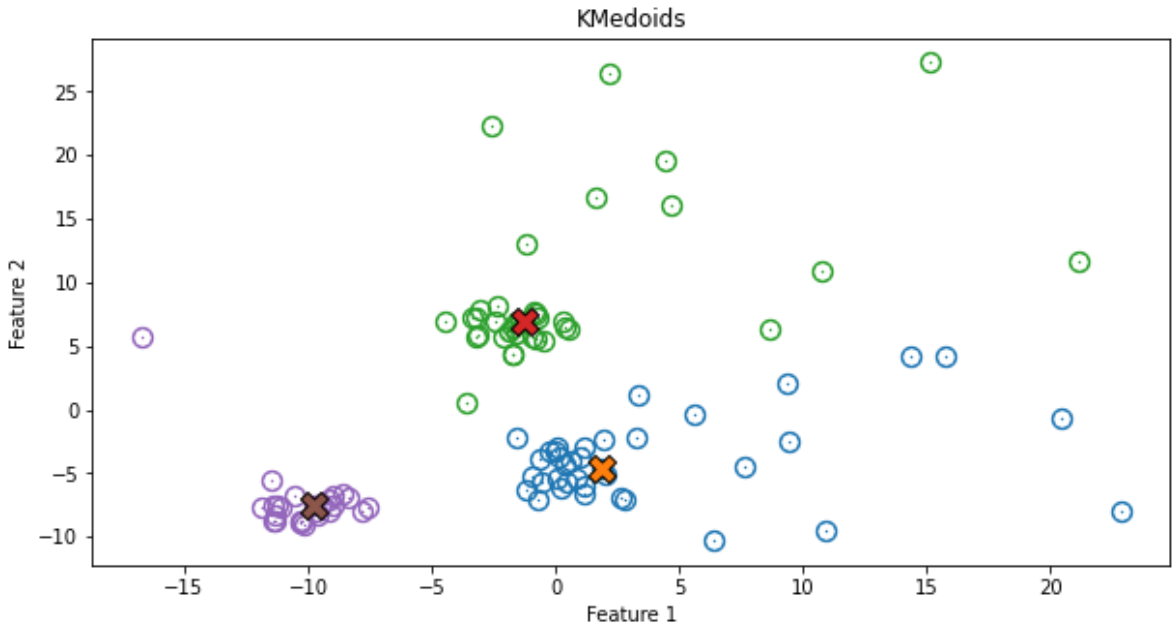
```
## Kmedoids plot

for i in range(len(centers)):
    X_c = X_noisy[members == i, :]
    plt.scatter(X_c[:, 0], X_c[:, 1], s=2, lw=10)
    plt.scatter(
        X_noisy[centers[i], 0],
        X_noisy[centers[i], 1],

        alpha=1.0,
        s=200,
        marker="X",
        edgecolor="black",
    )
plt.title('KMedoids')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.plot()
```

Out[8]:

```
[]
```



In [ ]:

```
You can see that KMedoids also performed well on a noisy dataset. It is better than the out
KMeans , which we saw earlier.
```